🚀 Day 9 Summary

Today I learned:

- Protocol creation
- Protocol methods
- Protocol properties
- Protocol implementation
- Protocol vs Inheritance
- Real-world iOS Protocol usage
- Flexible architecture design

🔷 Protocol Implementation

```
protocol CarDrive {
    func drive()
}

class Car: CarDrive {

    func drive() {
        print("Driving...")
    }
}

let bmw = Car()
bmw.drive()
```

🔷 Important Concept

Protocols only define requirements.

Protocols do NOT provide implementation.

They say:

```
"You MUST implement this."
```

🔷 Protocol Properties

Protocols can also require properties.

Read-Only Property

```
protocol HasName {
    var name: String { get }
}
```

`{ get }` → readable only

Read & Write Property

```
protocol HasAge {
    var age: Int { get set }
}
```

`{ get set }` → readable and writable

🔷 Protocol Example with Property

```
protocol HasName {
    var name: String { get }
}

class Student: HasName {

    var name: String

    init(name: String) {
        self.name = name
    }
}
```

🔷 Real iOS Example

```
struct ContentView: View
```

`View` is a protocol.

SwiftUI heavily uses Protocols.

🔷 Protocol vs Inheritance

Inheritance	Protocol
Reuse parent class	Define behavior rules
Tight connection	Flexible
Only classes	Struct + Class
"IS-A" relationship	"CAN-DO" relationship

🔷 Real Understanding

Inheritance:

```
Dog IS Animal
```

Protocol:

```
Bird CAN Fly
```

🔷 Practice Example – CanEat

```
protocol CanEat {
    func eat()
}

class Human: CanEat {

    func eat() {
        print("People can eat...")
    }
}
```

🔷 Practice Example – Game Player

```
protocol Properties {

    func canRun()
    func canAttack()
}

class GamePlayer: Properties {

    func canRun() {
        print("Player is Running")
    }

    func canAttack() {
        print("Player Attacked")
    }
}
```

🎯 Important Interview Questions

1️⃣ What is a Protocol in Swift?

A Protocol is a blueprint that defines required properties and methods.

2️⃣ Why do we use Protocols?

Protocols create flexible and reusable architecture.

They help separate:

- What should happen
- How it happens

---

3 What is the difference between Protocol and Inheritance?

Inheritance describes:

- What object IS

Protocol describes:

- What object CAN DO

---

4 Can Structs adopt Protocols?

Yes. Both Structs and Classes can adopt protocols.

---

5 What does `{ get }` mean in Protocols?

The property is read-only.

---

6 What does `{ get set }` mean?

The property can be read and modified.

---

7 Do Protocols provide implementation?

No. Protocols only define requirements.

Classes/Structs provide implementation.

---

8 Why are Protocols important in iOS development?

Protocols are heavily used in:

- SwiftUI
- Delegation
- Networking
- MVVM architecture
- API handling
- Reusable components

⭐ Quick Revision

- Protocol = Rules / Contract
- Protocol defines requirements
- Struct/Class implements requirements
- Protocols improve flexibility
- Protocols work with Structs & Classes
- `{ get }` = read-only
- `{ get set }` = read + write
- Protocol ≠ Inheritance